SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4: Articulation (BL4, BL5)**	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	10% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	NANUBALA MADHUSUDHAN	**Reg. No.:**	192425303
Section / Batch:	Slot-C	**Date:**	25/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze the given questions thoroughly before answering.
- Compare multiple aspects such as methods, results, advantages, and limitations clearly.
- Critically evaluate the strengths, weaknesses, and implications with proper justification.
- Present meaningful insights, interpretations, and well-supported conclusions from the comparisons.

Question Type	Focus Area	Questions
Comparative Analysis	Comparative analysis of the Principle of Optimality (Dynamic Programming) and the Greedy Choice Property for resource allocation optimization.	Q1

SECTION A – Comparative Analysis

Q22. Principle of Optimality vs Greedy Choice Property (Conceptual Comparison)

Consider a resource allocation problem and analyze it using the Principle of Optimality (DP) and Greedy Choice Property. Compare how each principle influences algorithm design. Identify conditions where both hold, evaluate limitations when one fails, and discuss the impact on correctness and optimality.

Code of Conduct

I, NANUBALA MADHUSUDHAN (192425303), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: NANUBALA MADHUSUDHAN

Q22. Principle of Optimality vs Greedy Choice Property

(Conceptual Comparison)

1. Problem Statement

A logistics company must allocate a limited resource — say, a fleet budget of ₹100 (in ₹10 lakh units) — across five regional warehouses (W1–W5), each yielding a different profit per unit invested. The goal is to allocate the budget to maximize total profit. This resource allocation problem is analyzed using two design paradigms:

- Principle of Optimality — the foundation of Dynamic Programming (DP)
- Greedy Choice Property — the foundation of Greedy algorithms

Then we compare:

- How each principle shapes algorithm design
- Conditions under which both hold simultaneously
- What happens when one principle fails
- Impact on correctness and optimality
- Time and space complexity
- Insights on choosing the right paradigm

2. Example — Resource Allocation Problem

Total Budget = 10 units. Each warehouse has an investment cost and a profit table (profit is not simply proportional to cost, so exhaustive combination matters):

Warehouse	Units Invested	Profit	Profit / Unit	Remark
W1	6	12	2.0	High cost, moderate ratio
W2	5	11	2.2	Best ratio
W3	5	11	2.2	Best ratio
W4	4	8	2.0	Moderate
W5	3	5	1.67	Lower ratio

3. Principle of Optimality — Dynamic Programming Approach

Idea

The Principle of Optimality states: an optimal solution to a problem contains within it optimal solutions to its subproblems. This means the best allocation of the full budget must be built from the best allocations of smaller budgets — so DP solves and stores every relevant subproblem before combining them.

Recurrence Relation

```
DP[i][b] =
  { DP[i-1][b]                                  if cost[i] > b
  { max( DP[i-1][b],
         DP[i-1][b-cost[i]] + profit[i] )        otherwise
```

where i = warehouse index, b = budget remaining.

DP Table (Budget 0–10)

Warehouse	0	1	2	3	4	5	6	7	8	9	10
None	0	0	0	0	0	0	0	0	0	0	0
W1=6/12	0	0	0	0	0	0	12	12	12	12	12
W2=5/11	0	0	0	0	0	11	12	12	12	12	12
W3=5/11	0	0	0	0	0	11	12	12	12	12	22
W4=4/8	0	0	0	0	8	11	12	12	12	19	22
W5=3/5	0	0	0	5	8	11	12	13	16	19	22

Maximum achievable profit = 22, using budget 10.

Optimal Allocation

Tracing the table backward gives the optimal set:

- W2 (5 units, profit 11) + W3 (5 units, profit 11) = 10 units, profit 22
- W2 (5) + W4 (4) + 1 unit unused = profit 19 — suboptimal, leaves budget idle
- W1 (6) + W4 (4) = 10 units, profit 20 — suboptimal
- Best combination: W2 + W3 = 10 units → profit 22, the highest value the table reaches at budget 10

DP guarantees this is the best possible combination because it systematically builds the optimal solution for budget 10 from the already-optimal solutions for every smaller budget (0 through 9) — the Principle of Optimality in action.

4. Greedy Choice Property — Greedy Approach

Idea

The Greedy Choice Property states: a globally optimal solution can be reached by making a locally optimal (best-looking) choice at each step, without reconsidering it later. Applied here, the greedy strategy picks the warehouse with the best profit-per-unit ratio first.

Steps

- Sort warehouses by profit/unit ratio, descending: W2 (2.2), W3 (2.2), W1 (2.0), W4 (2.0), W5 (1.67)
- Step 1: Pick W2 → cost 5, profit 11, remaining budget = 5
- Step 2: Pick W3 → cost 5, profit 11, remaining budget = 0
- Step 3: No budget left; W1, W4, W5 cannot be selected

Greedy total profit = 11 + 11 = 22, using the full budget of 10 units.

In this particular instance, greedy happens to match the DP result. This is not guaranteed in general, as shown next.

5. Example Where the Greedy Choice Property Fails

Consider a smaller, sharper case with budget = 10 units:

Warehouse	Cost	Profit	Ratio
P	6	14	2.33
Q	5	11	2.2
R	5	11	2.2

Greedy Execution

- Greedy sorts by ratio: P (2.33), Q (2.2), R (2.2)
- Picks P → cost 6, profit 14, remaining budget = 4
- Neither Q (cost 5) nor R (cost 5) fits in the remaining 4 units
- Greedy total = 14, and 4 units of budget go unused

DP Execution

- DP checks every feasible combination within budget 10: {P}, {Q}, {R}, {Q,R}
- P alone = 6 units, profit 14
- Q + R = 10 units, profit 22 — uses the full budget and yields more profit
- Best DP result = 22

Here DP strictly outperforms greedy: DP = 22 versus Greedy = 14. Greedy picked the item with the best individual profit-to-cost ratio, but that single choice consumed enough budget to block the better two-item combination. Because greedy never reconsiders a decision once made, it cannot recover from this — it commits to P and stops, while DP correctly identifies that skipping P in favor of Q and R produces a strictly better result. This is the Greedy Choice Property failing: the locally best choice was not part of the globally optimal solution.

6. Why the Greedy Choice Property Can Fail

The Greedy Choice Property only holds when picking the locally best option at each step is provably part of some globally optimal solution — this must be mathematically proven for the specific problem (as it is for, e.g., activity selection or Huffman coding). For general discrete resource allocation:

- Choices are interdependent — selecting one warehouse changes which combinations remain feasible for the rest of the budget
- Greedy never reconsiders a decision once made, so it cannot recover from a bad early commitment
- There is no guarantee that a locally optimal ratio leads to a globally optimal total profit

The Principle of Optimality, by contrast, always holds for problems with optimal substructure — which is why DP guarantees correctness even when greedy cannot.

7. Algorithm Steps

A. Dynamic Programming — Principle of Optimality

- Define state DP[i][b] = maximum profit using first i warehouses with budget b
- Base case: DP[0][b] = 0 for all b

- Transition: for each warehouse, either exclude it (carry forward DP[i-1][b]) or include it (DP[i-1][b-cost[i]] + profit[i]) if it fits
- Take the maximum of the two choices at every state
- Trace back through the table to recover the selected warehouses

```
Allocate(warehouses, B):
  for b = 0 to B: DP[0][b] = 0
  for i = 1 to n:
    for b = 0 to B:
      if cost[i] <= b:
        DP[i][b] = max(DP[i-1][b],
                       profit[i] + DP[i-1][b-cost[i]])
      else:
        DP[i][b] = DP[i-1][b]
  return DP[n][B]
```

B. Greedy — Greedy Choice Property

- Compute profit/cost ratio for each warehouse
- Sort warehouses in descending order of ratio
- Iterate through the sorted list; select a warehouse if its cost fits the remaining budget
- Deduct its cost from the remaining budget and move to the next
- Stop when no remaining warehouse fits

```
GreedyAllocate(warehouses, B):
  sort warehouses by (profit/cost) descending
  remaining = B
  selected = []
  for each warehouse w:
    if w.cost <= remaining:
      selected.append(w)
      remaining -= w.cost
  return selected
```

8. Correctness Analysis

Dynamic Programming (Principle of Optimality)

DP is correct whenever the problem exhibits optimal substructure — i.e., the Principle of Optimality holds. Because DP explicitly evaluates both including and excluding each item at every budget level, and reuses optimal answers to smaller subproblems, it is guaranteed to find the true optimum. This guarantee does not depend on the specific data — it holds for any cost/profit values.

Greedy (Greedy Choice Property)

Greedy is correct only when the Greedy Choice Property AND optimal substructure both hold and can be proven for the problem class (e.g., fractional knapsack, minimum spanning tree, Huffman coding). For discrete 0/1 resource allocation, no such proof exists in general — greedy's local ratio-based choice can conflict with the globally optimal combination, so correctness is not guaranteed and must be checked case by case.

9. Conditions Where Both Properties Hold Simultaneously

Both the Principle of Optimality and the Greedy Choice Property hold together when:

- The problem has optimal substructure (required for DP) AND
- A provable exchange argument shows the locally best choice is always part of some optimal solution (required for greedy)

Examples where both hold:

- Fractional Knapsack — items can be split, so the highest ratio item can always be taken first without loss
- Minimum Spanning Tree (Kruskal's / Prim's) — the cheapest safe edge is always extendable to an optimal tree
- Huffman Coding — merging the two least-frequent nodes is always optimal
- Activity Selection — choosing the activity with the earliest finish time is always safe

In these cases, greedy is preferred because it achieves the same optimal result as DP with far lower time and space cost.

10. Limitations When One Principle Fails

Situation	Consequence
Optimal substructure fails	DP cannot be applied meaningfully; subproblem solutions cannot be combined into a global optimum
Greedy Choice Property fails but optimal substructure holds	Greedy gives a fast but possibly incorrect/suboptimal answer; DP is required for guaranteed correctness
Both fail	Neither paradigm applies directly; problem may need brute force, branch-and-bound, or approximation/heuristic methods
Both hold	Greedy is preferred — same optimal result, much lower cost

11. Complexity Comparison

Feature	Dynamic Programming	Greedy
Guiding Principle	Principle of Optimality (optimal substructure)	Greedy Choice Property (local best choice)
Optimality	Guaranteed	Guaranteed only if property holds
Decision Scope	Considers all subproblem combinations	Considers only the current best option
Time Complexity	O(nB)	O(n log n) for sorting
Space Complexity	O(nB)	O(n)
Reconsideration	Yes — via subproblem table	No — decisions are final
Best suited for	Discrete, interdependent allocation problems	Problems with proven exchange property

Here: n = number of resources/items, B = total budget.

12. Impact on Correctness and Optimality

- DP's reliance on the Principle of Optimality makes correctness a mathematical guarantee — it never needs to be re-verified per instance
- Greedy's correctness is instance- and problem-class-dependent; using greedy without proving the exchange property risks silently wrong results in production systems
- In resource allocation, this matters directly: a greedy budget allocator could under-allocate to profitable smaller projects because it locks in early high-ratio picks, costing the organization real profit
- DP trades this risk for higher time and memory cost, which becomes significant as the budget or item count grows large

13. Real-World Resource Allocation Applications

1. Capital Budgeting

Firms allocate limited capital across projects with different costs and returns — DP guarantees the optimal project mix; greedy gives a fast approximate mix for large project lists.

2. Cloud Resource Scheduling

Allocating CPU/memory quotas across competing jobs to maximize throughput, where greedy heuristics are used for speed at scale despite occasional suboptimality.

3. Marketing Budget Allocation

Distributing an advertising budget across channels with diminishing or interdependent returns — DP suits precise campaigns; greedy suits real-time bidding decisions.

4. Workforce/Task Allocation

Assigning limited staff-hours across projects with different value contributions — a classic discrete allocation problem.

5. Portfolio Investment

Selecting a discrete set of investment options under a fixed capital limit to maximize expected return.

14. Key Insights

- Insight 1: The Principle of Optimality is a structural property of the problem — either it holds or it doesn't — and it is the precondition for DP to even be applicable.
- Insight 2: The Greedy Choice Property is a stronger, additional guarantee; problems that have it are a special subset of problems that have optimal substructure.
- Insight 3: When both hold, greedy is strictly preferable — it reaches the same optimum with less computation.
- Insight 4: When only the Principle of Optimality holds, DP is necessary — greedy may look reasonable but silently return a suboptimal allocation.
- Insight 5: For very large-scale resource allocation, a hybrid approach (greedy for an initial fast allocation, refined by local search or partial DP) is common in practice.
- Insight 6: Choosing the right paradigm requires first proving which structural properties the problem satisfies — not just picking the algorithm that is easier to implement.

15. Final Comparison

Parameter	Principle of Optimality (DP)	Greedy Choice Property
Basis	Optimal substructure of subproblems	Provable local-to-global choice
Decision making	Combines all subproblem solutions	Makes one irrevocable choice per step
Global optimization	Yes	Only if property proven
Optimal solution	Guaranteed	Not guaranteed generally
Time	O(nB)	O(n log n)
Space	O(nB)	O(n)
Implementation	More complex	Simple
Large-scale use	Computationally expensive	Fast and practical

Conclusion

The Principle of Optimality is the fundamental structural property that makes Dynamic Programming applicable and guarantees a globally optimal solution, because it allows a problem to be broken into subproblems whose optimal solutions combine into the overall optimum. The Greedy Choice Property is a stronger and rarer condition: it guarantees that a sequence of locally best, never-reconsidered choices also reaches the global optimum. When both properties hold — as in fractional knapsack or minimum spanning tree problems — greedy is the superior choice, matching DP's optimality at a fraction of the time and space cost. However, for discrete, interdependent resource allocation problems where the Greedy Choice Property cannot be proven, greedy can silently produce a suboptimal allocation, making Dynamic Programming the only reliable approach despite its higher computational cost. Selecting the right paradigm therefore requires first identifying which of these two properties the problem actually satisfies, rather than defaulting to whichever algorithm is simpler to code.